

操作系统实验指导书

实验二：内存管理

北京邮电大学 计算机学院

目录

一、实验目的.....	1
二、实验要求及内容.....	1
2.1 内存分配-跟踪双线程实验	1
2.2 内存分配-跟踪双进程实验	2
三、实验形式.....	3
四、实验设备环境.....	3
五、实验原理及步骤.....	3
5.1 内存分配-跟踪双线程实验	4
5.2 内存分配-跟踪双进程实验.....	5
六、编译及运行方法.....	7
6.1 内存分配-跟踪双线程实验	7
6.2 内存分配-跟踪双进程实验	7
6.3 实验结果说明	8
七、实验报告书写要求.....	8

一、实验目的

在本次实验中，需要从不同的侧面了解 openEuler 的虚拟内存机制。在 openEuler 操作系统中，可以通过一些 API 操纵虚拟内存。主要需要了解以下几方面：

- openEuler 虚拟存储系统的组织
- 如何控制虚拟内存空间
- 如何追踪进程内存使用情况
- 如何使用共享内存进行进程间通信
- 如何结合使用共享内存和信号量实现进程同步
- 了解 openEuler 内存映射机制
- 详细了解与内存相关的 API 函数的使用

二、实验要求及内容

2.1 内存分配-跟踪双线程实验

创建一个包含两个线程的进程：线程 t1 和线程 t2，进程 t1 通过执行一系列内存操作来模拟内存分配活动，线程 t2 用于跟踪线程 t1 的内存行为，两个线程通过信号量进行同步。内存分配线程 t1 负责实验目的中控制虚拟内存空间，内存跟踪线程 t2 负责实验目的中的追踪进程内存使用情况。

线程 t1 执行的内存操作类型包括：分配虚拟内存、写入虚拟内存、释放虚拟内存、锁定物理内存、解锁物理内存，这些操作是实验目的中控制虚拟内存空间的多种具体方式。可以将内存操作编号保存到输入文件中，线程 t1 从输入文件中读取内存操作并执行，输入文件的每一行包括如下内容：

- 内存操作的类型
- 内存操作的起始地址（可以是自定义的相对地址）

- 内存操作的页数
- 特定内存操作需要的额外信息，如分配虚拟内存操作需要的访问权限信息

设计的实验需覆盖以下操作序列（可以通过多个输入文件做多个实验来覆盖）：

- 分配一块虚拟内存区域，然后打印该区域上的部分或全部内容
- 分配一块虚拟内存区域，然后对该区域执行写入操作
- 释放一块虚拟内存区域，然后打印该区域上的部分或全部内容
- 释放一块虚拟内存区域，然后对该区域执行写入操作

在程序开始时，线程 **t2** 需打印操作系统的页面大小、内存总量、可用内存大小等内存信息；线程 **t1** 执行完每个内存操作后需打印该内存操作的类型、起始地址和页数，然后线程 **t2** 需打印进程已使用的物理内存大小、虚拟内存大小等信息。详细分析说明实验结果反映了什么问题或现象。

2.2 内存分配-跟踪双进程实验

由于 2.1 中跟踪线程与内存分配线程位于同一进程中，进程的内存使用情况也受跟踪线程本身影响，现改用两个进程完成 2.1 中的要求。要求创建两个进程 **p1** 和 **p2**，进程 **p1** 通过执行一系列内存操作来模拟内存分配活动，进程 **p2** 用于跟踪进程 **p1** 的内存行为。两个进程通过共享内存进行通信，通过将信号量保存在共享内存中来实现进程同步，这体现了使用共享内存进行进程间通信以及结合使用共享内存和信号量实现进程同步的实验目的。

进程 **p1** 执行的内存操作与 2.1 中相同。

先运行程序 **p2**，然后在另一个控制台运行程序 **p1**。在 **p1** 运行前，进程 **p2** 需打印操作系统的页面大小、内存总量、可用内存大小等内存信息；在 **p1** 开始运行后、开始一系列内存操作前，**p2** 打印 **p1** 的 **PID** 以及 **p1** 已使用的物理内存大小、虚拟内存大小和共享内存大小等信息；在 **p1** 每个内存操作后（包括 **p1** 完成最后一个内存操作后，**p1** 退出前），**p2** 打印该内存操作的类型、起始地址和页数以及 **p1** 已使用的物理内存大小、虚拟内存大小和共享内存大小等信息。通过将信号量保存在共享内存中来实现进程同步，这体现了实验目的中结合使用共享内存和信号量实现进程同步的要求。

进程 p1 和 p2 通过共享内存进行通信，传送的信息可以包括信号量、PID 以及内存操作的类型、起始地址和页数等信息，这体现了使用共享内存进行进程间通信的实验目的。可以在程序 p1 和 p2 的命令行参数中指定共享内存对象的标识符。

三、实验形式

小组实现。每组 1 名组长，注意明确分工，平衡工作量。

四、实验设备环境

服务器：

- openEuler 20.03 (LTS), Linux version 4.19.90-2003.4.0.0036.oe1.aarch64
- gcc (GCC) 7.3.0

虚拟机：

- openEuler 20.03 (LTS), Linux version 4.19.90-2003.4.0.0036.oe1.x86_64
- gcc (GCC) 7.3.0

五、实验原理及步骤

openEuler 内核给每个进程都提供了一个独立的虚拟地址空间，并且这个地址空间是连续的，进程就可以很方便地访问内存。进程的虚拟地址空间被划分为许多大小可变的虚拟内存区域（Virtual Memory Area, VMA）。一个虚拟内存区域由一组连续的具有相同属性（如访问权限）的页组成。

在 openEuler 中，可以通过头文件<sys/mman.h> 中的 mmap, munmap, mlock, munlock, shm_open, shm_unlink 等实现以页为单位的虚拟内存管理方法。这是实验目的中要求了解与使用的内存相关的 API 函数。内存分配线程 t1 通过（1）~（5）实现中控制虚拟内存空间的实验目的，内存跟踪线程 t2 通过（6）实现追踪进程内存使用情况的实验目的。

5.1 内存分配-跟踪双线程实验

线程 t1 和 t2 的伪代码如下：

P1	P2
<pre>while (从输入文件读取内存操作) { P(sem2) 执行内存操作 打印内存操作信息; V(sem1) } finished=true</pre>	<pre>打印操作系统内存信息; while (!finished) { P(sem1); 打印进程的内存使用信息; V(sem2) }</pre>

(1) 分配虚拟内存

```
void *mmap(void *addr, size_t length, int prot, int flags, int fd,  
off_t offset);
```

`mmap()` 在调用进程的虚拟地址空间创建一个新的文件映射或匿名映射。当参数 `fd` 为某文件的描述符时，创建一个新的文件映射，将该文件映射到进程的地址空间，通过修改地址空间的内容来修改文件内容；当 `fd=0` 时，创建一个新的匿名映射。

可以通过创建匿名映射（`flags=MAP_ANON, fd=0, offset=0`），来创建一个 VMA 或扩大一个已有的 VMA，如初次调用

```
mmap(addr, length, PROT_READ|PROT_WRITE|PROT_EXEC, MAP_ANON, 0, 0)
```

即创建一个起始地址为 `addr` 附近的一页的边界地址、大小为 `length` 个页的虚拟内存区域，该区域的内存保护策略为可读、可写可执行

（`PROT_READ|PROT_WRITE|PROT_EXEC`）。函数执行成功后返回分配的虚拟内存区域的首地址。

`mmap()` 是实验目的中要求了解的 openEuler 内存映射机制的一个主要 API 函数。

(2) 写入虚拟内存

`mmap()` 执行后，不会立即分配物理内存，只是使得分配的这部分虚拟内存在此后可以被有效访问。通常在写入后才分配内存。如果想要分配物理内存，需要使这部分内存的每页都变“脏”（可通过赋值使页变“脏”）。

（3）释放虚拟内存

```
int munmap(void *addr, size_t length);
```

`munmap()` 删除以 `addr` 为起始地址、大小为 `length` 字节的地址范围内的映射，使此后对该范围内的地址的引用无效。

`addr` 需要为页大小的倍数。

（4）锁定物理内存

```
int mlock(const void *addr, size_t len);
```

`mlock()` 会将调用进程地址空间内的以 `addr` 为起始地址、大小为 `len` 字节的区域锁到 RAM 中，防止这部分内存页被调度到交换空间（swap space）。

（5）解锁物理内存

```
int munlock(const void *addr, size_t len);
```

与锁定物理内存相反，`munlock()` 可以释放锁定。

（6）监控内存

`openEuler` 提供了很多监控内存使用情况的方式。

`/proc/[pid]` 文件系统下提供了进程的详细信息，包括内存使用信息，其中 `/proc/[pid]/stat` 文件中就提供了进程使用的虚拟内存大小（VSZ, Virtual memory size）和物理内存大小（RSS, Resident Set Size）。

5.2 内存分配-跟踪双进程实验

进程 `p1` 和 `p2` 的伪代码如下：

P1	P2
打开一个已有共享内存对象; 向共享内存中写入 PID; V(sem1); while (从输入文件读取内存操作) { P(sem2) 执行内存操作 将内存操作信息写入共享内存 V(sem1) } sleep(10);	打印操作系统内存信息; 新建一个共享内存对象; 初始化共享内存中的信号量 sem1=0, sem2=0; P(sem1); 从共享内存中读取 P1 的 PID; 打印 P1 的 PID 和内存使用信息; V(sem2); while (1) { P(sem1); 从共享内存读取内存操作信息; 打印内存操作信息; 打印 P1 的内存使用信息; V(sem2) }

代码中 P2 创建共享内存对象，P1 打开该共享内存对象，之后两者便可读写同一块共享内存，进行进程间的通信。通过把信号量存储在共享内存中，通过 P、V 操作控制信号量（即读写共享内存中的变量），便可实现进程同步。这是实验目的中使用共享内存进行进程间通信以及结合使用共享内存和信号量实现进程同步的具体实现方式。

（1）创建共享内存对象

```
int shm_open(const char *name, int oflag, mode_t mode);
```

shm_open 创建或打开一个共享内存对象，返回 1 个文件描述符。可以将返回的文件描述符作为上述 mmap() 函数的 fd 参数，来调用函数 mmap()，这样将会把该内存共享对象映射到进程的地址空间，之后就可以通过修改地址空间中的内容来修改共享内存中的内容。这是实验目的中要求了解的 openEuler 内存映射机制的一个具体体现。

(2) 移除共享内存对象

```
int shm_unlink(const char *name);
```

与 `shm_open` 相反, `shm_unlink` 移除当前进程上该共享内存对象的映射, 当所有进程都取消对该共享内存对象的映射后, 才释放共享内存对象。

六. 编译及运行方法

6.1 内存分配-跟踪双线程实验

将输入文件 `input.txt` 与代码文件 `memory-op.c` 放在同一路径。

编译: `gcc memory_op.c -o memory_op -l pthread`

运行: `./memory_op`

6.2 内存分配-跟踪双进程实验

设程序 `p1` 和 `p2` 的源代码分别是 `mm-op1.c` 和 `mm-op2.c`, 共享内存对象标识符为 `/myshm`。那么,

程序 `p1` 和 `p2` 编译命令分别是:

```
gcc mm-p1.c -o mm-p1 -lrt -pthread
```

```
gcc mm-p2.c -o mm-p2 -lrt -pthread
```

在一个控制台执行 `p2` (`/myshm` 为自定义的内存对象标识符, 也可以用其他任意字符串):

```
./mm-p2 /myshm
```

然后在一个新的控制台执行 `p1` (可以通过新建 `SSH` 会话)

```
./mm-p1 /myshm
```

6.3 实验结果说明

实验结果分析需要根据实际的实验结果数据进行详细分析，包括进程已使用的虚拟内存、物理内存大小等内存信息的变化。详细分析说明实验结果反映了什么问题或现象。

七、实验报告书写要求

学生的实验报告内容要与本指导书内容一致，要有：

实验目的：描述本次实验任务、目的；

实验要求及内容：描述本次实验具体要求，实验的具体内容，另外写出实验原理、整体思想，可给出相应原理图及整体框架图；

实验设备环境：描述本次实验的实验环境；

实验步骤（原理步骤、编译运行方法、实验结果说明）：首先写明实验的具体步骤，描述每个步骤对应的原理、流程图，写明实验的具体编译运行方法，最后展示实验结果，需要从理论分析和实际结果分析两方面分析对比实验结果，得出结论，会出现什么问题，如何解决；

实验心得体会：实验报告最后需要总结和写出心得体会，完成本次实验编写代码的时间、上机调试的时间、编程工具遇到的问题、编程语言遇到的问题、总结本次实验。